CNENGR 116
Fundamentals of Digital Electronics

Lecture 6 - Arithmetic Circuits

Dr Zhongming Zhang

Zhongming.zhang@lancaster.ac.uk

School of Engineering

Lancaster University

Science and Technology at Lancaster

Lancaster University College at Beijing Jiaotong University

Objectives

- Binary arithmetic and two's complements
- Half adder and full-adder circuits
- Comparator circuits

1. Binary Addition

- 0 + 0 = 0 carry 0
- 0 + 1 = 1 carry 0
- 1 + 0 = 1 carry 0
- 1 + 1 = 0 carry 1
- 1 + 1 + 1 = 1 carry 1

when the sum exceeds 1, carry a 1 over to the next-more-significant column.

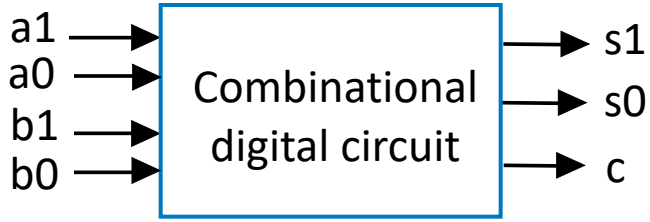
```
                0
A:15      1  1  1  1
B: 6    + 0  1  1  0
= 21    ------------
                   1

                1  0
          1  1  1  1
        + 0  1  1  0
        ------------
             0  1

             1  1
       1  1  1  1
     + 0  1  1  0
     ------------
       1  0  1

             1
       1  1  1  1
     + 0  1  1  0
     ------------
     1  0  1  0  1
```

1. Binary Addition

- Adding two *N*-bit binary numbers
  - e.g., 2-bit adder

```
    a1  a0
+   b1  b0
-----------
c   s1  s0
```

- Can we design a logic circuit to do the binary addition for a typical *N*-bit binary number?



Inputs				Outputs		
a1	a0	b1	b0	c	s1	s0
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	1	0
0	0	1	1	0	1	1
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	1
0	1	1	1	1	0	0
1	0	0	0	0	1	0
1	0	0	1	0	1	1
1	0	1	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	0	1	1	0	0
1	1	1	0	1	0	1
1	1	1	1	1	1	0

1. Binary Addition

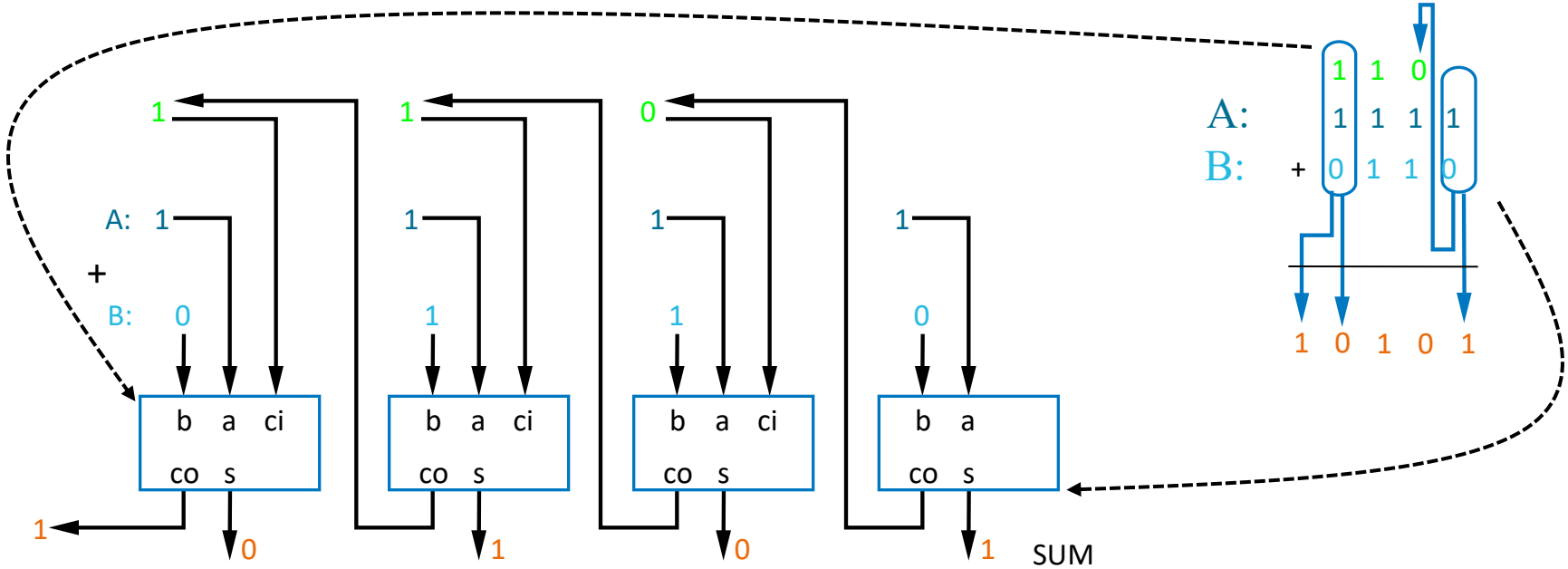
- Truth table too big
  - 2-bit adder's truth table shown
    - Has $2^{(2+2)}$ = 16 rows
  - 5-bit adder: $2^{(5+5)}$ = 1024 rows
  - 16-bit adder: $2^{(16+16)}$ = ~4 billion rows
  - 32-bit adder: 2^{64} = ~1.8×10^{19} rows

Inputs				Outputs		
a1	a0	b1	b0	c	s1	s0
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	1	0
0	0	1	1	0	1	1
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	0	1	1
0	1	1	1	1	0	0
1	0	0	0	0	1	0
1	0	0	1	0	1	1
1	0	1	0	1	0	0
1	0	1	1	1	0	1
1	1	0	0	0	1	1
1	1	0	1	1	0	0
1	1	1	0	1	0	1
1	1	1	1	1	1	0

- Big truth table with numerous 1s and 0s yields a large number of logic gates

1.1. Half-Adder

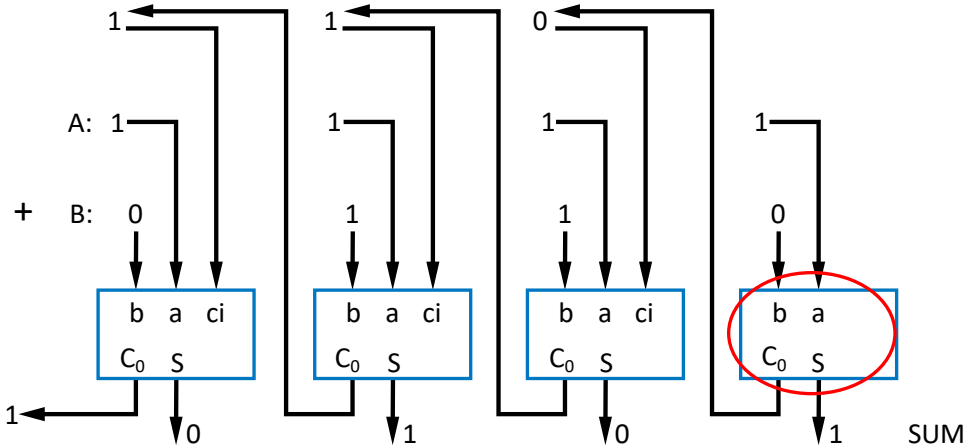
- Solution: create a component for each column
  - Performs an addition on that column's bits to generate sum and carry bits



1.1. Half-Adder

- **Half-adder**: Adds 2 bits and generates a sum and a carry
- Design using combinational design process
  - **Step 1 -** Capture the function

Inputs		Outputs	
a	b	C_0	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

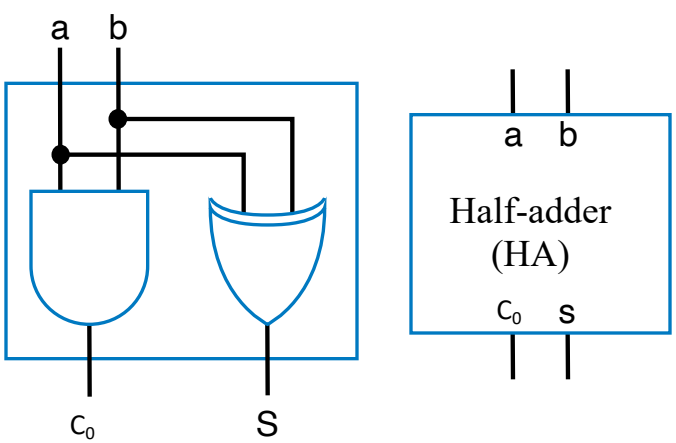


Full Adder Half Adder

1.1. Half-Adder

- **Step 2a -** Create equations
  - $C_0 = a.b$
  - $S = \bar{a}.b + a.\bar{b} = a \oplus b$

- **Step 2b -** Implement as circuit

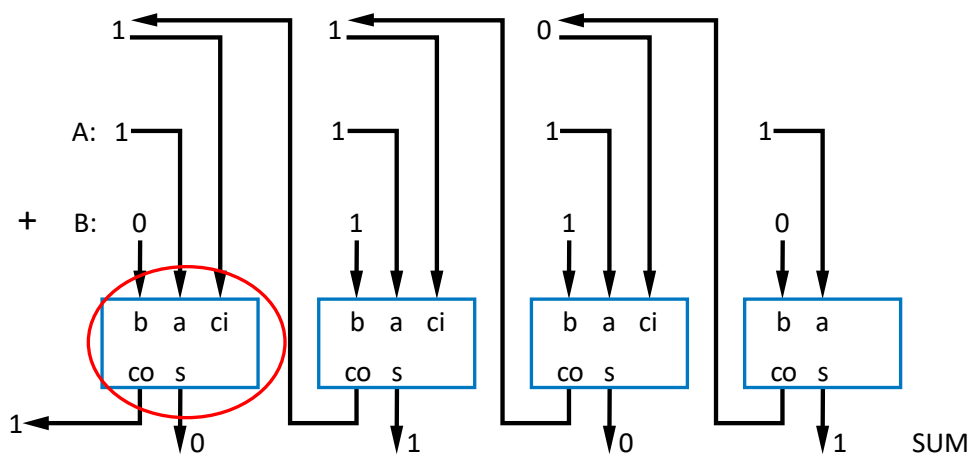


Inputs		Outputs	
a	b	C_0	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

1.2. Full-Adder

- **Full-adder**: Adds 3 bits and generates a sum and carry
- Design using combinational design process
  - **Step 1 -** Capture the function

Inputs			Outputs	
a	b	Ci	C_0	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Full Adder

Half Adder

1.2. Full-Adder

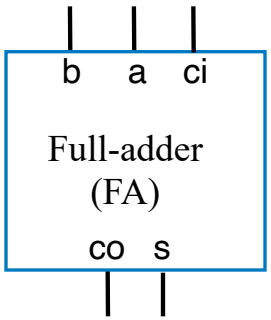
- **Step 2a -** Create equations
  - $C_0 = \bar{a}bc_i + a\bar{b}c_i + ab\overline{c_i} + abc_i$
  - $C_0 = ab(c_i + \overline{c_i}) + c_i(\bar{a}b + a\bar{b})$
  - $\boldsymbol{C_0 = ab + c_i(a \oplus b)}$

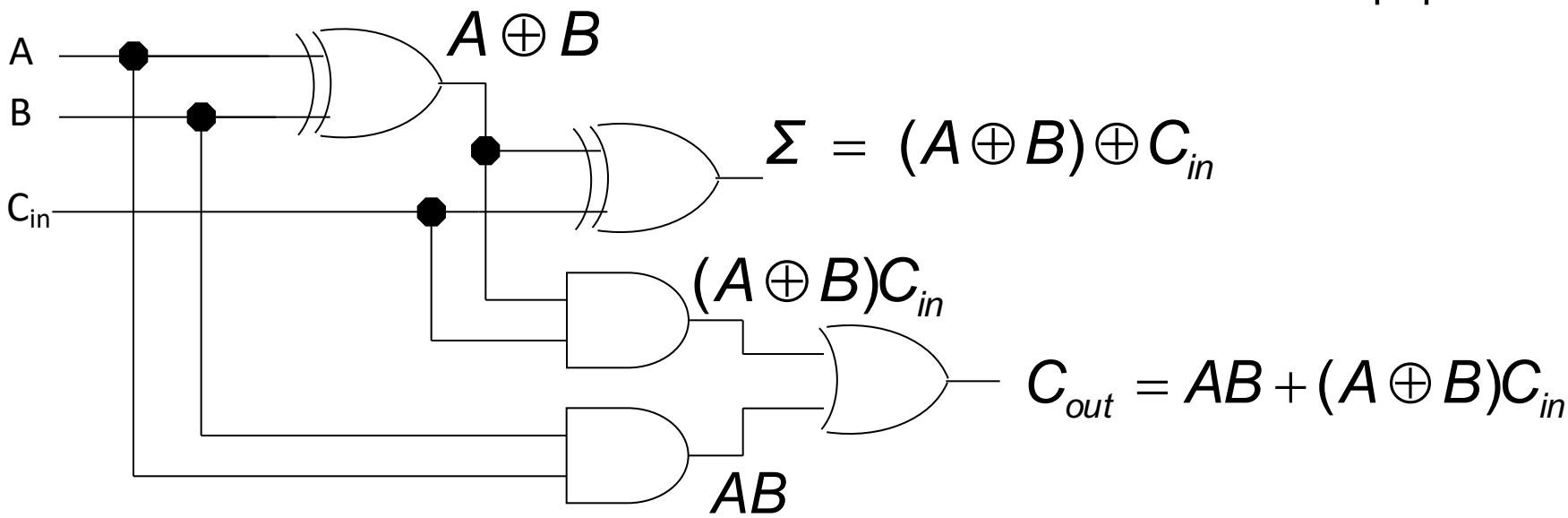
  - $S = \bar{a}\bar{b}c_i + \bar{a}b\overline{c_i} + a\bar{b}\overline{c_i} + abc_i$
  - $S = \bar{a}(\bar{b}c_i + b\overline{c_i}) + a(\bar{b}\overline{c_i} + bc_i)$
  - $S = \bar{a}(b \oplus c_i) + a\overline{(b \oplus c_i)}$
  - $\boldsymbol{S = a \oplus b \oplus c_i}$

Inputs			Outputs	
a	b	ci	Co	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

1.2. Full-Adder

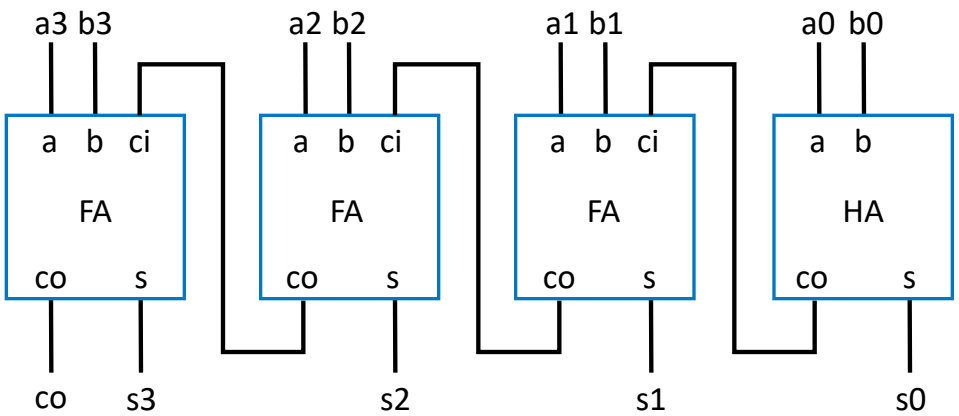
- **Step 2b -** Implement as circuit
  - $C_{out} = ab + c_{in}(a \oplus b)$
  - $S = a \oplus b \oplus c_{in}$

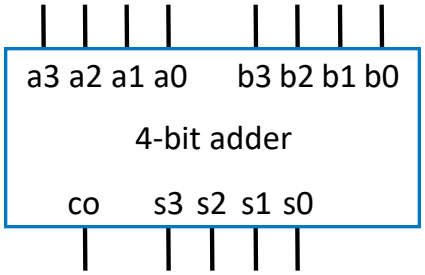




1.3. Carry-Ripple Adder

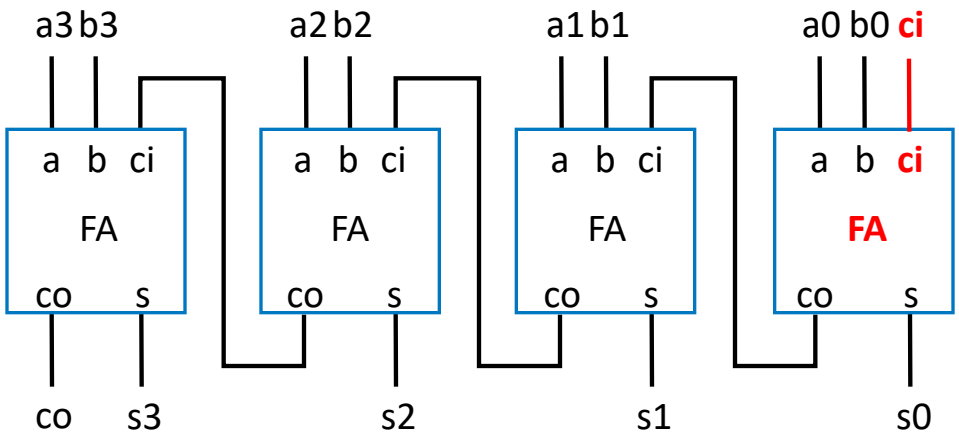
- **Carry-ripple adder:** Combining a half-adder and full-adders to build an adder that adds like we would by hand
- **E.g. 4-bit adder:** Adds two 4-bit numbers, generates 4-bit "sum" plus 1-bit "carry out"

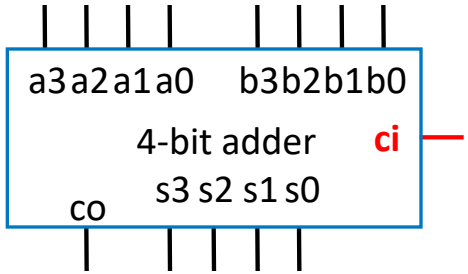




1.3. Carry-Ripple Adder

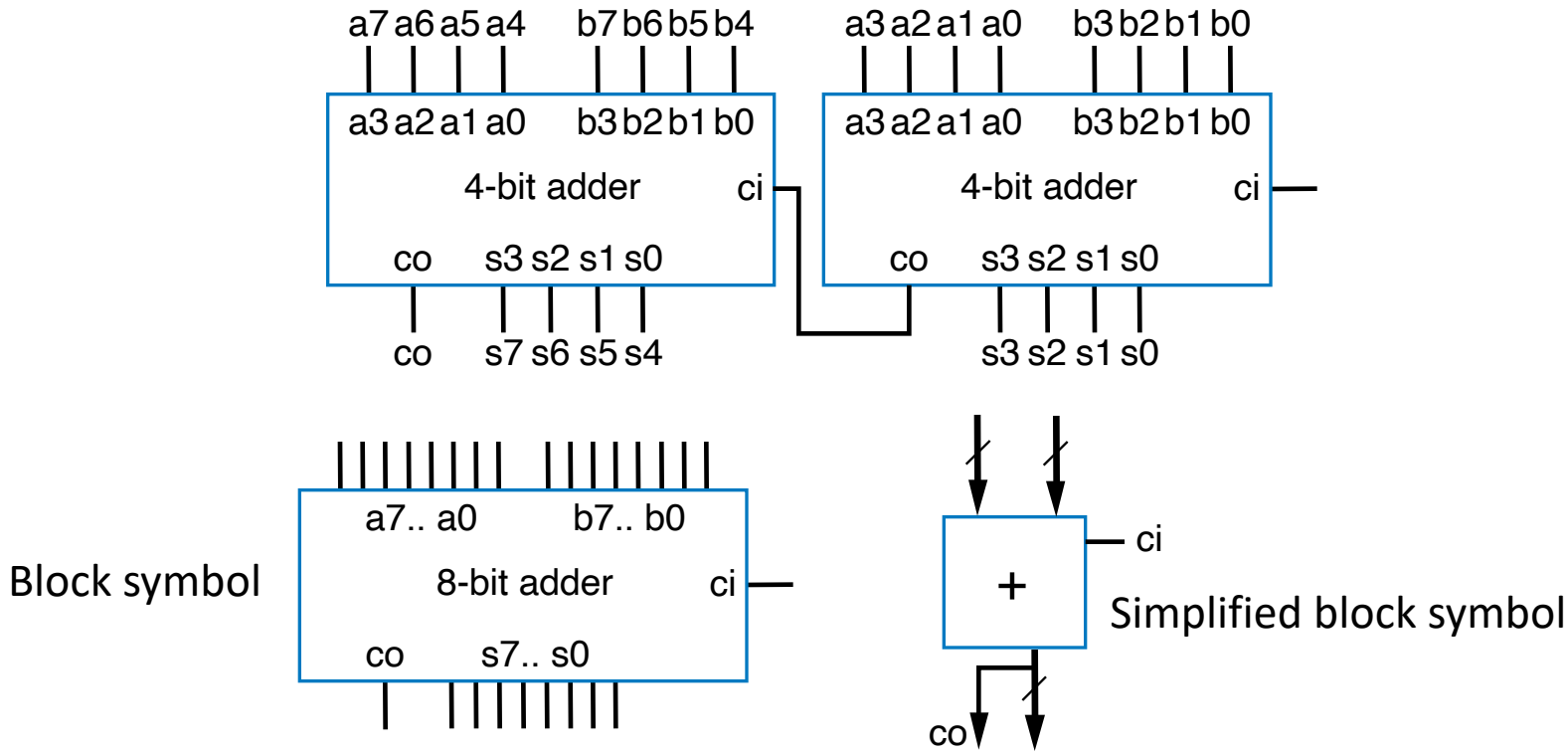
- Using full-adder instead of half-adder for first bit, we can include a “carry in” bit in the addition
  - Useful later when we connect smaller adders to form bigger adders





1.4. Cascading Adders

- Building any size adder



2. Binary Subtraction

- Subtraction
  - 0 – 0 = 0 borrow 0
  - 0 – 1 = 1 borrow 1
  - 1 – 0 = 1 borrow 0
  - 1 – 1 = 0 borrow 0

when 1 is subtracted from 0, borrow 1 from columns on the left, then 0 turns to '10' for the next-lower-significant column.

1st Column

$$\begin{array}{rcccc} & & & 0 & \\ & 1 & 0 & \cancel{1} & 10 \\ - & 0 & 1 & 1 & 1 \\ \hline & & & & 1 \end{array}$$

2nd Column

$$\begin{array}{rcccc} & 0 & 1 & 10 & \\ & \cancel{1} & \cancel{1}0 & \cancel{1} & 0 \\ - & 0 & 1 & 1 & 1 \\ \hline & & & 1 & 1 \end{array}$$

3rd Column

$$\begin{array}{rcccc} & 0 & 1 & & \\ & \cancel{1} & \cancel{1}0 & 1 & 0 \\ - & 0 & 1 & 1 & 1 \\ \hline & & 0 & 1 & 1 \end{array}$$

4th Column

$$\begin{array}{rcccc} & 0 & & & \\ & \cancel{1} & 0 & 1 & 0 \\ - & 0 & 1 & 1 & 1 \\ \hline & 0 & 0 & 1 & 1 \end{array}$$

2.2. Two's Complement

- Two's Complement
  - E.g. Complement of a 4-bit binary number is the difference of that number and 10000
  - More generally for a binary number with n bits, the complement is the difference between it and 2^n
- Example: Suppose we have 4 bits
  - Two's complement of 0011 is 10000-0011=1101.
  - An easier method: just invert all bits and add 1.
  - The two's complement of 0011 is 1100+1=1101 It works!

2.2. Two's Complement

- Two's complement can be used to represent negative numbers.
- Left most bit; 0 indicates a positive number, 1 indicates a negative number.

Positive numbers				Negative numbers
0	0000	→	1000	×
1	0001	→	1111	-1
2	0010	→	1110	-2
3	0011	→	1101	-3
4	0100	→	1100	-4
5	0101	→	1011	-5
6	0110	→	1010	-6
7	0111	→	1001	-7
×	1000	→	1000	-8

2.3. Class Exercise

- Assume that we are working with 4-bit binary number.

A) What is the two's complement of 0101?

1010+1=1011

A quick check: 0101+1011=10000

B) In two's complement format which decimal number is represented by 1110?

We need just to take two's complement of the number again

1110 → -(0001+1) = -0010 = -2

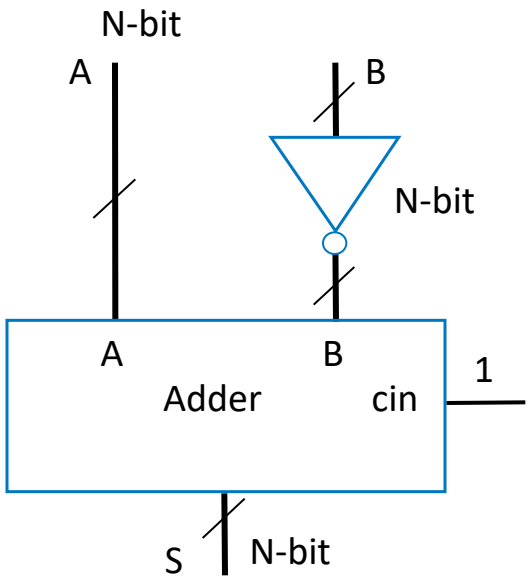
2.4. Subtractors

- Building a subtractor with an adder using two's complement
  - A – B = A + (– B)

    = A + (two's complement of B)

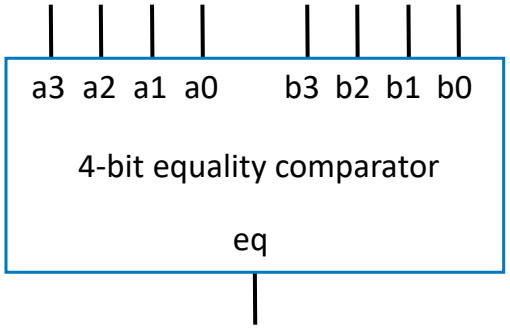
    = A + (one's complement of B + 1)

    = A + inverting bits of (B) + 1
- To build a subtractor using an adder
  - Invert bits of B and setting carry-in to 1
  - The remainder is the first N-bit of output



3. Comparators

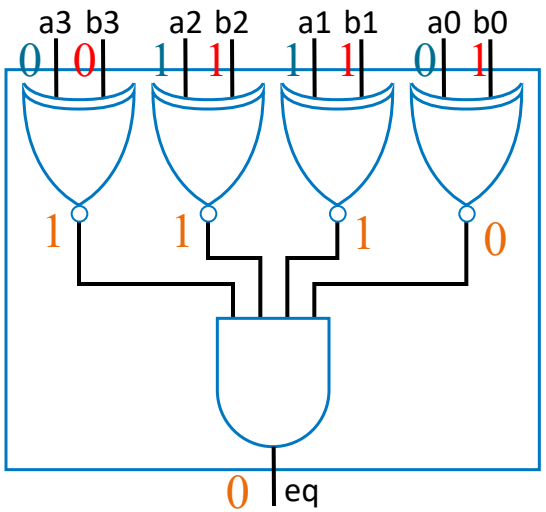
- **N-bit equality comparator:** Outputs 1 if two *N*-bit numbers are equal
- Design using combinational design process
- **Step 1 -** Capture the function
  - E.g. 4-bit equality comparator with inputs A and B
  - a3 = b3 and a2 = b2 and a1 = b1 and a0 = b0
  - Two bits are equal if both 1 or both 0



3. Comparators

- **Step 2a -** Create equations
  - $eq = (a_3b_3 + \overline{a_3}\overline{b_3}).(a_2b_2 + \overline{a_2}\overline{b_2}).(a_1b_1 + \overline{a_1}\overline{b_1}).(a_0b_0 + \overline{a_0}\overline{b_0})$
  - $eq = (\,).(\,).\ (\,).(\,)$
- **Step 2b -** Implement as circuit

$0110 = 0111$?



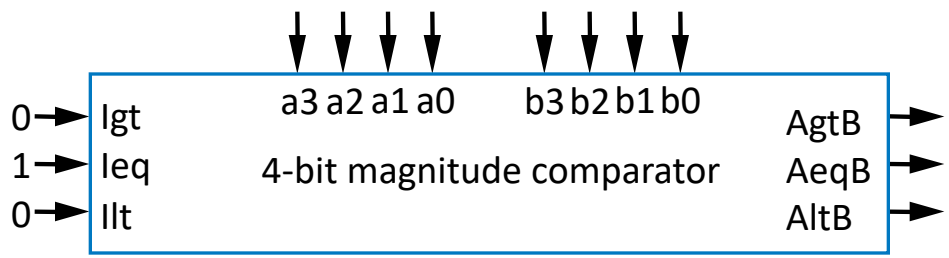
3.1. Magnitude Comparator

- **N-bit magnitude comparator**: Two N-bit inputs A and B, outputs whether A>B, A=B, or A<B, for
- **Step 1 -** Capture the function
  - E.g. 4-bit magnitude comparator with inputs A and B
  - First compare a3 and b3. If equal, compare a2 and b2 and so on.
  - Stop if comparison not equal.

A=1011 B=1001

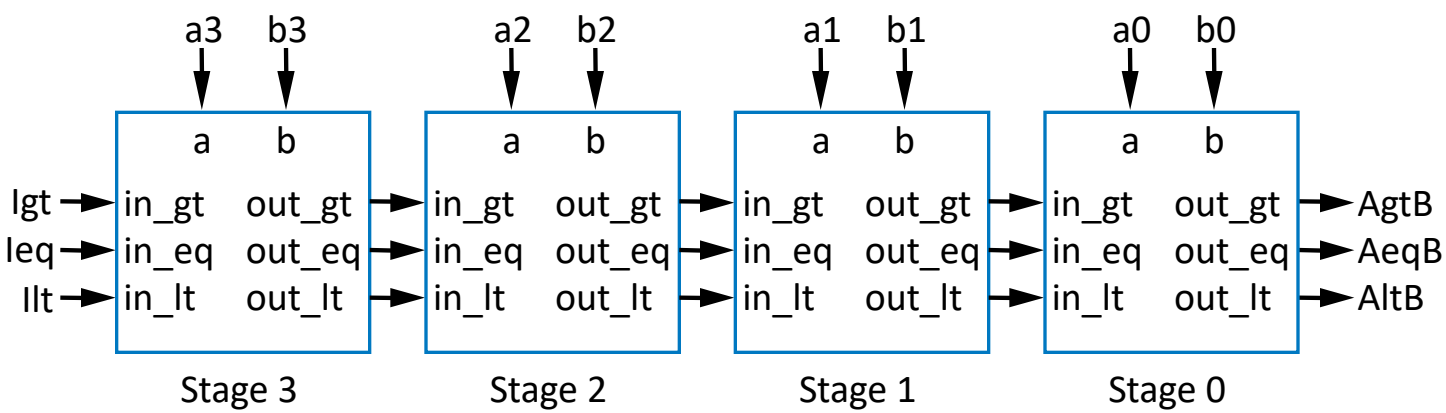
1011	1001	Equal
1011	1001	Equal
1011	1001	**Not equal**

so A > B



3.1. Magnitude Comparator

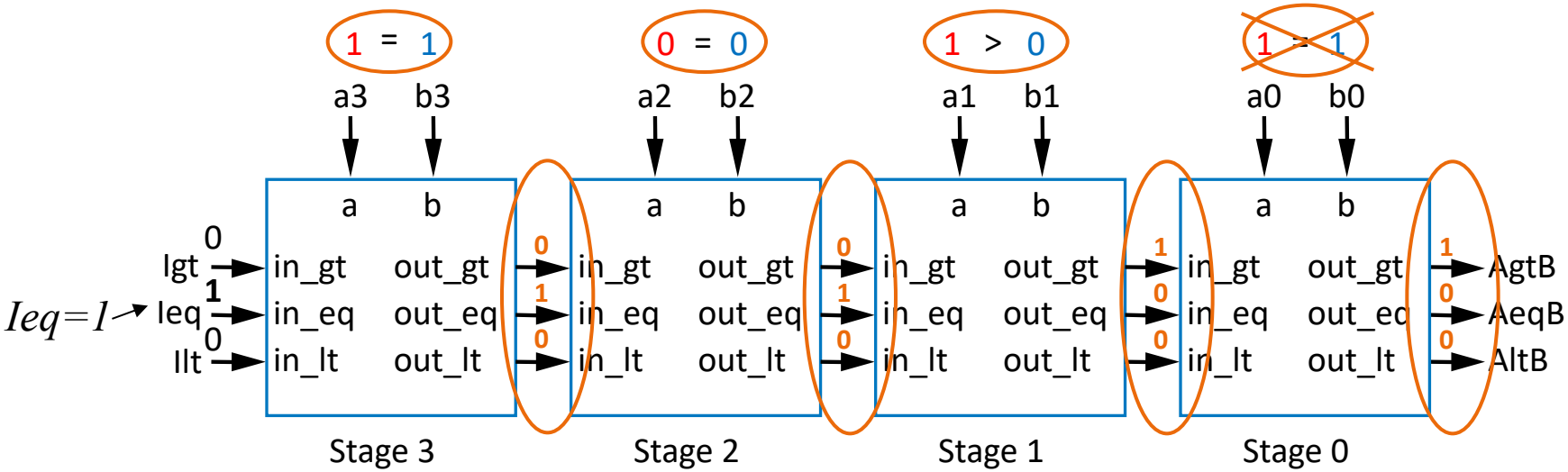
- **Step 2a, 2b -** Create equations and implement as circuit
  - Each bit pair called a *stage*
  - Each stage has 3 inputs taking results of higher stage, outputs new results to lower stage



3.1. Magnitude Comparator

- How does it work?

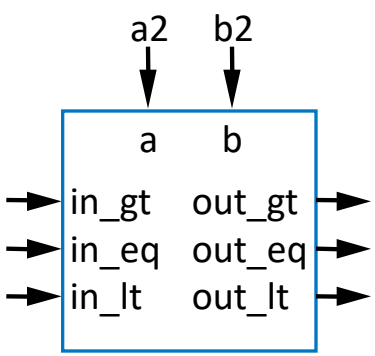
1011 = 1001 ?

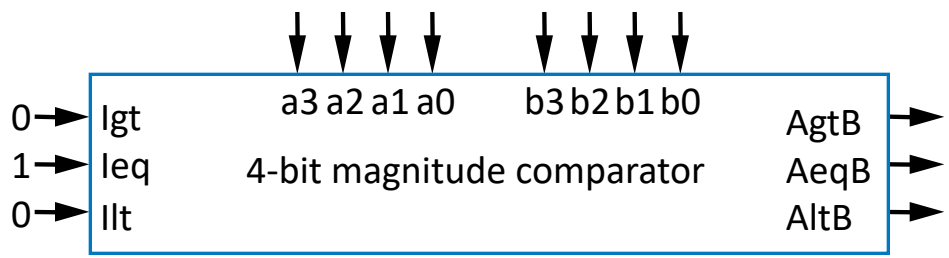


3.1. Magnitude Comparator

- Each stage:
  - $out_gt = in_gt + (in_eq \,.\, a \,.\, \bar{b})$
  - $out_lt = in_lt + (in_eq \,.\, \bar{a} \,.\, b)$
  - $out_eq = in_eq \,.\, (\overline{a \oplus b})$

Have a go at building these three circuits!





Summary

- Half-adder and full-adder circuits to do binary addition
- Carry-ripple adder to add two *n*-bit binary numbers
- Two's complement method and binary subtractor
- Equality comparator and magnitude comparator circuits